ROME Hierarchical RTL + iVerilog/Covered Coverage Integration

Sahil Dunani · NetID sd6429 · LLM4ChipDesign — HW ROME + Coverage · October 26, 2026

1. LLM model and settings

All LLM calls were served by **OpenAI gpt-4o-mini** via the OpenAI Python SDK, wrapped by the ROME Conversation class with model_type="ChatGPT". The API key is pulled from Google Colab *Secrets* as OPENAI_API_KEY; it is never written into the notebook or the repository. Temperature and *top_p* are the defaults supplied by ROME. The PPA-optimizer cell originally hard-coded model_type="Claude"; it was switched to "ChatGPT" so a single key powers both the hierarchy-generation loop and the PPA refinement loop.

2. Chosen hierarchy (Part II — Option A)

The new hierarchy is a 4-level ripple-carry adder tree: **half_adder** → **full_adder** → **adder4** → **adder8**. Each level is strictly built from instances of the preceding level, so ROME's bottom-up loop can finish each module before the next one depends on it:

• **half_adder** — 1-bit a,b → sum,cout, pure combinational.
• **full_adder** — two half-adders plus an OR of the two half-carries.
• **adder4** — four full-adders chained through cin → cout.
• **adder8** — two adder4s chained; exposes the top-level 8-bit a,b,cin → sum,cout interface used by the coverage pipeline.

3. Final module prompts given to ROME

ROME takes [filename, natural-language description, interface]. The descriptions handed to hier_gen() for the adder design are:

half_adder — "Half-adder. Combinational logic that adds two 1-bit inputs *a* and *b*. Output *sum* = *a* ^ *b* and *cout* = *a & b*. No clock, no reset, no instances of other modules."

full_adder — "Full-adder. Combinational 1-bit adder with inputs *a*, *b*, *cin* and outputs *sum*, *cout*. Must be built by instantiating two *half_adder* modules and ORing the two carry outputs. No behavioural + at the top level."

adder4 — "4-bit ripple-carry adder. Inputs *a[3:0]*, *b[3:0]*, *cin*; outputs *sum[3:0]*, *cout*. Must instantiate four *full_adder* modules; carry chains from bit 0 to bit 3."

adder8 — "8-bit ripple-carry adder. Inputs *a[7:0]*, *b[7:0]*, *cin*; outputs *sum[7:0]*, *cout*. Must instantiate two *adder4* modules so the low 4 bits feed carry into the high 4 bits."

4. iVerilog + Covered integration

A run_coverage(tb_src, tag) helper was added to the notebook. For every candidate testbench it (a) writes the TB to coverage/tb_<tag>.v, (b) compiles with iverilog, (c) runs vvp, (d) strips the $comment block that Icarus 13 emits (Covered 0.7.10 can't parse it), (e) scores a CDD, (f) emits summary and detail reports, and (g) parses the four percentages (line, toggle 0→1, toggle 1→0, logic) into a shared coverage_history list.

The commands reproduce the coverage_exp example from the course repo and are the same ones used in the *final verification* cell:

```
iverilog -g2001 -Wall -s adder8_tb -o adder8.vvp \
    half_adder.v full_adder.v adder4.v adder8.v coverage/tb_final.v
vvp adder8.vvp
covered score -t adder8 -i adder8_tb.dut -v … -vcd clean.vcd -o adder8.cdd
covered report -m ltc -d s -o coverage_summary.rpt
covered report -m ltc -d v -i -o coverage_detail.rpt
```

5. Co-improvement iterations (RTL vs TB kept separate)

Design rule: RTL and TB were never changed in the same iteration. The ROME hierarchy loop produced correct RTL for all four modules on iteration 0, so the frozen RTL was carried into the coverage loop, and every subsequent

iteration touched only the testbench.

Iter.	Changed	What changed	Why
0 (RTL)	RTL	hier_gen() generated half_adder, full_adder, adder4, adder8	ROME verilog_loop passed iverilog + sanity TB; RTL frozen.
baseline	TB	5 hand-written vectors on adder8	Minimal TB; measures where coverage actually falls short.
iter_1	TB	LLM-added vectors driving a and b to 8'hFF, 8'h00, 8'hAA, 8'h55, plus a 1→0 corner cases	Fix the toggle[] seen in baseline detail report.
iter_2	TB	LLM adds 5 more targeted vectors (random alternating patterns, walking-one)	Keep 100% across all four metrics and show the loop is stable.

6. Coverage history (adder8, measured by Covered 0.7.10)

Iteration	# tests	Line	Toggle 0→1	Toggle 1→0	Logic
baseline	5	100.0%	100.0%	66.0%	100.0%
iter_1	10	100.0%	100.0%	100.0%	100.0%
iter_2	15	100.0%	100.0%	100.0%	100.0%

The single uncovered item at baseline was *toggle 1→0* on the upper bits of sum, b, and intermediate sum_high/sum_low (bits never driven back to 0 from an all-ones state). The iter_1 LLM prompt was fed the raw Covered detail block above and asked to generate additional vectors that drive those bits back to 0 — the single all-FF-to-all-00 transition it added closed every remaining toggle gap. iter_2 kept 100% to show the loop converges and does not regress.

7. Reproducing

Open rome_submission.ipynb in Colab, add an OPENAI_API_KEY secret, then *Runtime → Run all*. The notebook installs iverilog, covered and the OpenAI SDK, clones ROME, runs mux hier_gen + PPA (Part I), runs adder hier_gen (Part II), runs the coverage baseline + two iterations (Part III), and writes coverage/coverage_history.json plus rome_submission_artifacts.zip containing the generated RTL, every TB, the Covered summary/detail reports, and the coverage history (Part IV).